Spam Email Detection Using Naive Bayes Classifiers

Machine Learning Report

April 25, 2026

1 Introduction

Email spam detection is a critical application of Natural Language Processing (NLP) and supervised learning. This report details the implementation of various Naive Bayes classifiers to distinguish between legitimate (ham) and unsolicited (spam) emails using a dataset of 5,728 labeled messages.

2 Dataset Description

The dataset consists of two columns: the raw email text and a binary indicator (1 for spam, 0 for ham).

- **Total Emails:** 5,728
- **Spam Messages:** 1,368
- **Ham Messages:** 4,360

The data was split into training and testing sets with a 50% ratio to evaluate model performance across different Naive Bayes variants.

3 Methodology

3.1 Data Preprocessing

To transform raw text into a format suitable for machine learning, several preprocessing steps were applied:

1. **Cleaning:** Removal of non-letter characters and conversion to lowercase.
2. **Tokenization:** Splitting text into individual word tokens.
3. **Stopword Removal:** Filtering out common English words (e.g., "the", "is") that do not contribute to classification.

4. **Lemmatization:** Reducing words to their base or dictionary form (e.g., "winning" becomes "win") using the NLTK WordNet Lemmatizer.

In [ ]
```
def preprocess_spam(text):
    # Clean: Remove non-letters & lowercase
    text = re.sub(r'[^a-zA-Z]', ' ', text).lower()
    # Tokenize: Split into words
    words = text.split()
    # Lemmatize & Remove Stopwords
    clean_words = [lemmatizer.lemmatize(w) for w in words
                   if w not in stop_words]
    return  .join(clean_words)
```

3.2 Feature Extraction

We utilized the `TfidfVectorizer` (Term Frequency-Inverse Document Frequency) to convert the cleaned text into numerical vectors. The feature space was limited to the top 5,000 most frequent terms to maintain computational efficiency while capturing the most discriminative words.

4 Model Evaluation

Four variants of the Naive Bayes algorithm were implemented and compared:

4.1 Comparison of Accuracy

The models demonstrated high performance across the board, with Complement Naive Bayes (CNB) achieving the highest accuracy.

Model	**Accuracy**
Gaussian Naive Bayes	94.90%
Multinomial Naive Bayes	96.54%
Bernoulli Naive Bayes	93.47%
Complement Naive Bayes	**97.94%**

Table 1: Model Performance Comparison

4.2 Detailed Classification Reports

Below is the execution output for the best performing model (Complement NB):

In [ ]

```
y_pred_cnb = cnb.predict(X_test)
print(Complement NB Accuracy:, accuracy_score(y_test, y_pred_cnb))
print(\nClassification Report:)
print(classification_report(y_test, y_pred_cnb))
```

Out:

```
Complement NB Accuracy:  0.9793994413407822
Classification Report:  precision recall f1-score support
0 0.99 0.98 0.99 2150 1 0.94 0.98 0.96 714
accuracy 0.98 2864 macro avg 0.97 0.98 0.97 2864 weighted avg 0.98 0.98 0.98
2864
```

5 Conclusion

The experimental results indicate that Naive Bayes classifiers are highly effective for spam detection tasks. While all models performed well (above 93% accuracy), the **Complement Naive Bayes** classifier proved to be the most robust, achieving an accuracy of **97.94%**. This suggests that handling class imbalances and normalized weights provides a significant advantage in text classification within this specific dataset.